# SAÉ S2.01 - Développement d'une application en Java

Système de gestion d'un cabinet médical

IUT de Lens - Département Informatique

2025-2026

IUT
Institut
Universitaire
Technologie
LENS

# Table des matières


Phase 1 - Modèle objet de l'application . . . 2
Introduction . . . 2
Livrables . . . 2
Fonctionnalités attendues . . . 2
Phase 2 - Implémentation de tests et ajout d'une interface graphique . . . 3
Configuration de départ . . . 3
Fonctionnement pendant la phase 2 de la SAÉ S2.01 . . . 3
Développement collaboratif . . . 4
Stratégie Git . . . 5
Trace d'exécution de l'application . . . 7
Modification de la configuration du projet . . . 8
Modification du code . . . 9
Écriture du code des fonctions de calcul. . . . 10
Mise en place des tests unitaires . . . 10
Modification de la configuration du projet . . . 10
Les tests unitaires à écrire . . . 11
Gestion de la documentation . . . 14
Réalisation de l'interface graphique (travail à réaliser l'après-midi) . . . 14
Affichage des différents éléments de l'application . . . 15
Création d'un « écran d'accueil » . . . 19

# Phase 1 - Modèle objet de l'application

Cette partie rappelle le travail demandé lors de la première phase de la SAÉ S2.01. Pour la seconde phase, vous pouvez passer directement à la section suivante.

## Introduction

L'objectif ici est de développer un système simple de gestion de dossiers médicaux pour un cabinet médical. Ce système permettra de gérer les informations des patients, des médecins et des rendez-vous. Il est impératif de tenir compte des exigences suivantes dans la réalisation de cette application :

- le principe d'encapsulation devra être respecté ;
- le mécanisme d'héritage devra être utilisé, notamment à travers la création de sous-classes ;
- au moins une énumération devra être utilisée ;
- l'ajout des méthodes *getters* et *setters* devra être réalisé en fonction du contexte ;
- toute classe devra contenir au moins un constructeur adapté ;
- toute classe devra contenir une méthode `toString()` et en fonction du contexte une méthode `equals()` ;
- l'ajout de commentaires devra être fait afin de faciliter la compréhension de votre code source.

L'implémentation réalisée devra uniquement suivre les approches vues en TD et en TP.

## Livrables

1. Le code source de votre application.
2. Un rapport d'au moins 10 pages décrivant les éléments suivants :
   - la liste des fonctionnalités implémentées dans votre programme et de celles qui ne le sont pas ;
   - une description rapide des différents éléments du programme (paquetages, classes, énumération,…) ;
   - l'approche à suivre pour tester les différentes fonctionnalités de votre application avec des exemples concrets (en particulier, le cas échéant, vous devez indiquer pour les différentes fonctionnalités les classes à exécuter) ;
   - la répartition des tâches entre les différents membres du groupe.

## Fonctionnalités attendues

L'application complète devra comporter les fonctionnalités suivantes :

1. Gestion des patients : créer, lire, mettre à jour et supprimer les informations des patients.
2. Gestion des médecins : enregistrer les médecins avec leurs spécialisations.
3. Gestion des médicaments : créer et lister des médicaments.
4. Gestion des rendez-vous : prendre, annuler et lister les rendez-vous entre patients et médecins.
5. Gestion des ordonnances : permettre aux médecins de créer des ordonnances pour les patients, en enregistrant les informations sur les médicaments prescrits, la posologie et la durée du traitement.
6. Module de recherche : implémenter des fonctionnalités de recherche pour trouver des patients ou des médecins par nom, spécialité ou autre critère.
7. Module de statistiques : afficher des statistiques (nombre de médecins, de patients, médicaments,…).
8. Toutes autres fonctionnalités que vous jugerez pertinentes.

## Phase 2 - Implémentation de tests et ajout d'une interface graphique

Pour la seconde phase de la SAÉ S2.01, nous allons compléter votre application de **Système de gestion de dossiers médicaux** développée dans la première phase de cette SAÉ.

### Configuration de départ

Nous vous avons préparé un projet de départ qui contient, à peu de chose près, le code de base de l'application demandée dans la phase 1.

Le diagramme de classes (sans les attributs et méthodes) est donné dans la figure suivante :

Ce projet de départ est disponible dans le dépôt GitLab de la SAÉ S2.01 : Gestion Cabinet Medical.

### Fonctionnement pendant la phase 2 de la SAÉ S2.01



L'un des membres du groupe aura pour tâche de créer un *fork* du projet de départ (*Fork* en anglais, *Créer une bifurcation* en français). Ce nouveau projet sur *Gitlab* contiendra le résultat de votre travail pour la phase 2.



Un **lien** vers ce projet devra être indiqué dans le cours *Moodle* SAE S2.01 Développement d'une application, dans le devoir Votre projet sur Gitlab. Vous pouvez le rendre dès la création de votre projet, ce lien étant indépendant des modifications que vous y apporterez au cours de la journée.

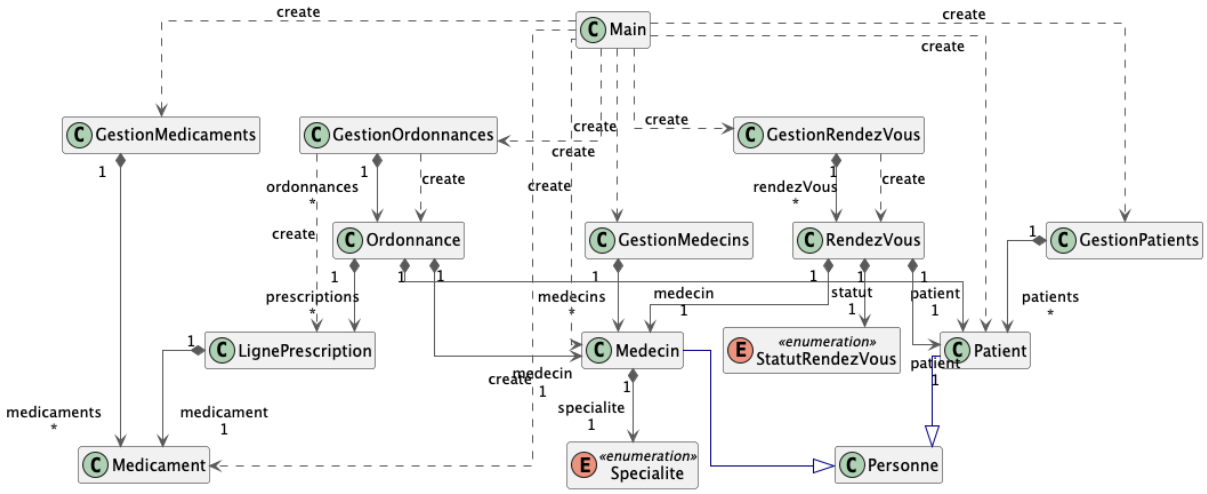



**FIGURE 1 –** Diagramme de classes

Le propriétaire du projet (le membre qui a créé le *fork*) devra autoriser les autres membres du groupe à modifier le projet sur *GitLab*. Pour cela, il devra les ajouter comme « *Maintainer* » du projet sur *GitLab*.

À partir de ce moment, chaque membre du projet pourra faire un clone du projet dans son propre espace de travail, et importer le projet dans son environnement de développement pour commencer à travailler dessus.

N'oubliez pas de mettre également les enseignants qui participent à cette SAÉ comme « *Reporter* » de votre projet sur *GitLab* afin que chacun d'entre eux puisse le récupérer pour l'évaluer :

- Fred Hemery
- Romain Wallon

**Attention :** Il est important que chaque membre du groupe puisse accéder au projet sur *GitLab* et l'importer dans son environnement de travail **AVANT 9h30** afin de respecter les délais fixés par ce sujet. Si vous avez des difficultés pour le partage du projet ou son import, **prévenez les enseignants de la SAÉ S2.01 dès que possible** pour que nous puissions vous aider à résoudre ces problèmes techniques sans impacter les délais de développement.

### Développement collaboratif

Le projet est à réaliser en groupe de 3 ou 4 étudiants. Chaque membre du groupe doit contribuer à différentes parties du projet, que ce soit la modification du code existant, le développement des tests

et le développement de l'interface graphique en *JavaFX*.

Chaque modification du projet est motivée par une tâche, un ticket ou un *work item* sur *GitLab*. La première chose à faire est d'identifier l'ensemble des tâches à effectuer pour la phase 2 avant de se lancer dans le développement. Il n'est pas toujours évident d'avoir une vision globale, l'énoncé doit donc vous permettre d'identifier ces tâches. **Vous devez donc le lire attentivement en intégralité afin de pouvoir rédiger ces tickets**. Nous allons vous demander d'effectuer dans un premier temps les tâches relatives à l'élaboration de tests, à la création de quelques fonctions de calcul ainsi que la mise en place d'un mécanisme de trace. Dans un second temps, vous pourrez vous attaquer à la partie interface graphique de l'application.

Ce premier découpage doit déjà vous aider à lister un certain nombre de tâches à réaliser, que vous pourrez ensuite détailler en sous-tâches plus précises.

Pour ne pas avoir de conflit de travail, il est important que chaque membre du groupe prenne en charge **un ticket à la fois**, et que les modifications soient poussées régulièrement sur *GitLab*, afin que les autres membres du groupe puissent les voir, les commenter et en bénéficier.

#### Stratégie Git

Pour assurer une collaboration efficace et une gestion de code propre, nous allons adopter quelques règles dans l'utilisation de *Git* et *GitLab*.

**Stratégie de branches** Nous utilisons une stratégie de branches *Git* qui permet de séparer les différentes fonctionnalités et de faciliter les revues de code. La branche `main` contient le code stable et déployable, tandis que la branche `dev` est utilisée pour l'intégration progressive des fonctionnalités développées par l'équipe. Chaque fonctionnalité ou tâche est développée dans une branche locale dédiée, qui doit suivre la convention de nommage décrite ci-dessous :

```
main                → code stable, déployable (protégée)
dev                 → intégration continue de l'équipe
├── feature/compute
├── feature/logging
├── feature/test
└── feature/ihm
```

**Stratégie de mise à jour du code partagé** La démarche à suivre par chaque membre du groupe pour mettre à jour proprement le code partagé est la suivante :

- chaque membre choisit un ticket (dans *GitLab*, on lui assigne la tâche), et aucun autre membre ne travaille sur cette tâche.
- le membre développe la fonctionnalité associée à l'issue sur une branche dédiée (ex : `feature/compute/coutOrdonnance` pour une fonctionnalité de calcul, `feature/logging/serviceMedecin` pour une fonctionnalité de trace, *etc.*).
- quand le membre considère que son code est correct (utilisation des outils d'analyse statique, tests unitaires, *etc.*) et que la fonctionnalité est prête, il va le fusionner sur la branche `dev` du projet. Plus précisément, il doit suivre les étapes suivantes :
  - ajouter les fichiers modifiés à l'index de *Git* (`git add` suivi des fichiers concernés),
  - faire un commit de sa branche de travail en mentionnant le ticket concerné (par exemple, pour la branche `feature/compute/coutOrdonnance` : `git commit -m "close #21 : modification de la fonction coutTotal terminée"`) pour être dans un état stable,
  - changer de branche locale et se placer sur la branche `dev` (`git switch dev`),
  - récupérer le code actuel de la branche `dev` sur *GitLab* et l'ajouter sur sa branche locale `dev` (`git pull origin dev`) afin de récupérer le travail des autres membres du groupe (si le membre a fait ses modifications sur une branche locale dédiée, il ne doit pas y avoir de conflit),
  - fusionner sa branche locale sur la branche `dev` locale (`git merge feature/compute/coutOrdonnance`),
  - gérer les conflits éventuels en local (il peut y avoir discussion avec les autres membres et faire un arbitrage final) qui doivent être minimes, si la répartition du travail est correcte,
  - pousser sa branche locale `dev` sur la branche `dev` du projet *GitLab* (`git push origin dev`).

En appliquant cette technique, les conflits sont gérés en local, la version partagée est toujours stable, et les autres membres du groupe peuvent bénéficier de la fonctionnalité développée dès qu'elle est prête.

Quand l'équipe considère que l'état de la branche `dev` est stable, elle peut être fusionnée dans la branche `main` pour être considérée comme une version finale du projet. C'est le « *responsable* » du groupe qui fait cette fusion.

Chaque membre du groupe peut alors récupérer la branche `main` du projet *GitLab* et la fusionner avec sa branche locale (`git pull origin main`).

Vous devez également indiquer dans le fichier `README.md` de votre projet la liste des tickets (étape de réalisation de votre projet). Pour chaque ticket, on devra identifier le membre de l'équipe qui l'a pris en charge et un *commit*, au moins, devra le confirmer dans le projet sur *GitLab*.

Exemple de tableau à compléter :

| Membre du groupe | Tâches réalisées | Ticket associé |
|---|---|---|
| Alice | création de la fonction coutGlobal | #8 |
| Bob | Gestion de la trace dans ServiceMedecin | #3 |
| Charlie | Correction bug dans la fonction coutGlobal | #5 |
| David | Tests unitaires, Documentation du projet | #7 |
| Bob | Interface graphique de la liste des patients | #7 |

Pour cela, lorsque vous ferez un *commit*, n'oubliez pas d'indiquer le numéro du ticket dans le message de votre *commit*. Par exemple :

```
1 git commit -m "Ticket #1 : Modif de la classe Patient"
```

Assurez-vous de vous **répartir les tâches de manière équitable** : chaque membre du groupe doit participer au développement, en écrivant **au moins une classe de tests et une vue de l'application avec son service**. Les notes des membres du groupe pourront être adaptées en fonction de la participation individuelle au développement.

Votre travail devra être déposé avant le **lundi 15 juin à 17h30**.

> **Information importante :** Le tableau ci-dessus que vous devez remplir dans le fichier `README.md` sera utilisé pour faire une évaluation individuelle. Il est donc important de le compléter en respectant les règles de nommage des tickets et des *commits* de votre projet pour les réalisés. Assurez-vous également que chaque membre du groupe réalise **au moins une tâche dans chacune des composantes essentielles** indiquées dans la grille d'évaluation.

**Respect des bonnes pratiques de développement** Dans le développent du projet, il vous est demandé de respecter les normes de nommage associées au langage *Java*. Il est aussi rappelé que l'évaluation de votre travail tiendra compte d'outils d'analyse de code statique comme *SonarQube*, et que s'il y a trop de manquements au respect de l'écriture du code cela pourra être pénalisé.

### Trace d'exécution de l'application

Le code fourni contient de nombreux appels à la fonction `System.out.println()` pour afficher des informations sur la console.

On peut distinguer son utilisation dans les fonctions de calcul et dans les fonctions d'affichage, et son utilisation dans la classe `Main` pour afficher des informations sur l'exécution de l'application.

Dans le cas de la classe `Main`, on peut considérer que c'est justifié : on souhaite avoir un aperçu du fonctionnement de l'application. Nous allons néanmoins utiliser un *logger* pour afficher les informations dans la console.

Par contre, dans les autres cas, l'utilisation de `System.out.println()` n'est pas justifiée, car elle rend le code plus difficile à lire et à maintenir, et elle ne permet pas de différencier les différents types d'informations affichées ( informations de calcul, informations d'affichage, *etc.*).

C'est pourquoi, dans la suite, nous allons remplacer les appels à `System.out.println()` par des appels à un *logger*, qui permettra de différencier les différents types d'informations affichées, et de les configurer pour qu'elles soient affichées ou non en fonction du contexte.

#### Modification de la configuration du projet

Pour pouvoir utiliser le logger, il faut d'abord configurer le projet pour qu'il puisse gérer ce type de dépendances. Pour cela, il faut ajouter les dépendances dans le fichier `build.gradle` pour que *Gradle* puisse les télécharger et les utiliser dans le projet. Ensuite, il faut ajouter un fichier de configuration pour le *logger*. **Pour éviter les conflits sur *Git*, assurez-vous que une seule personne du groupe effectue cette modification et les partage pour que les autres membres du groupe puissent utiliser ces nouvelles dépendances.**

Dans notre projet, nous allons définir trois sorties de log (`Appenders`) :

- une sortie vers la console pour la classe `Main`,
- une sortie vers la console pour les classes dans le package `fr.univartois.butinfo.sae.cabinet` autres que `Main`, et
- une sortie vers un fichier qui sera stocké dans un répertoire `logs`.

Le format de ces sorties est le suivant :

```
1  <PatternLayout pattern="[%-5level] %d{yyyy-MM-dd HH:mm:ss} %c{1} - %msg
      %n"/>
```

et

```
1  <PatternLayout pattern="%msg%n"/>
```

pour les sorties vers la console et la classe `Main`.

On distinguera deux *loggers* :

- un *logger* par défaut (*Root*) qui utilisera le niveau `INFO`, et

— un *logger* spécifique pour les classes dans le paquetage `fr.univartois.butinfo.sae.cabinet` qui utilisera le niveau `DEBUG`.

**Exercice 1 :**

Configurez votre projet pour pouvoir utiliser un *logger*. Pour cela :

1. Ajoutez les dépendances nécessaires dans le fichier `build.gradle`.
2. Ajoutez un fichier de configuration `log4j2.xml` pour le logger dans le dossier `src/main/resources`.
3. Testez que votre projet compile et que vous pouvez lancer l'application avec *Gradle*.

### Modification du code

L'infrastructure pour l'utilisation du mécanisme de trace étant en place, nous allons maintenant adapter le code pour utiliser le *logger*.

**Exercice 2 :**

Adaptation du code pour utiliser le *logger*.

1. Remplacez l'appel de la fonction `System.out.println()` par l'utilisation du logger dans les classes de votre projet. En adaptant le niveau en fonction du contexte (par exemple, `DEBUG` pour les informations de calcul, `INFO` pour les informations d'affichage, *etc.*).

**Exercice 3 :**

2. Modifiez la classe `Statistiques` :
    1. Renommez la fonction `void afficherTableauDeBord()` en `String getStatistiques()`
    2. Modifiez les instructions `System.out.println()` par un stockage dans une variable de type `StringBuilder` pour retourner une chaîne de caractères contenant les statistiques.
    3. Adaptez la classe `Main` pour afficher les statistiques à l'aide du *logger*.

**Exercice 4 :**

3. Modifiez la classe `Recherche`. Les fonctions recherches sont mal écrites, elles contiennent un appel à une fonction d'affichage (`afficherResultatsXXX()`) et renvoient le résultat de la recherche. La fonction de recherche n'a pas à afficher de résultat, elle peut éventuellement signaler un mauvais fonctionnement en cas d'erreur.
    1. Modifiez les fonctions de recherche pour qu'elles ne contiennent plus d'appel à une fonction d'affichage, qu'elles retournent uniquement le résultat de la recherche et qu'elles utilisent le *logger* en cas d'erreur.
    2. Supprimez les fonctions d'affichage de la classe.
    3. Adaptez la classe `Main` pour afficher les résultats de la recherche à l'aide du *logger* (vous pouvez vous inspirer des fonctions `afficherResultatsXXX()` pour vous aider).

### Écriture du code des fonctions de calcul.

Dans le projet, deux fonctions de calcul sont à implémenter :

— Dans la classe `Ordonnance`, la fonction `double getPrixTotal()` qui permet de calculer le coût total d'une ordonnance en fonction des médicaments prescrits et du taux de remboursement du patient.
— Dans la classe `RendezVous`, la fonction `prixConsultation()` qui permet de calculer le prix d'une consultation en fonction du temps de consultation, du taux horaire proposé par le médecin et du taux de remboursement du patient.

### Mise en place des tests unitaires

L'architecture du projet ne permet pas actuellement de mettre en place des tests unitaires. Nous allons ajouter les dépendances qui vont permettre d'en mettre en place.

#### Modification de la configuration du projet

Nous allons ajouter les dépendances pour l'utilisation de :

— *JUnit 5* pour les tests unitaires,
— *JUnit Platform* pour lancer les tests unitaires,

— *AssertJ* pour les assertions, et
— *Mockito* pour les mocks.

**Exercice 5 :**

1. Modifiez le fichier `build.gradle` pour pouvoir utiliser *JUnit 5*, *JUnit Platform*, *AssertJ* et *Mockito*. **Pour éviter les conflits sur *Git*, assurez-vous que une seule personne du groupe effectue cette modification et les partage.**
   — Il faut vous assurer que vous utilisez la même version pour `junit-jupiter` et `junit-platform-launcher`.
   — L'utilisation de *Mockito* demande une configuration particulière (voir le TP tests unitaire).
2. Testez que votre projet compile et que vous pouvez lancer les tests unitaires avec *Gradle*.

### Les tests unitaires à écrire

Les tests doivent être utiles. Dans le cadre de ce projet, nous allons ajouter des tests unitaires pour les classes modèles suivantes :

— `Ordonnance` et `RendezVous` qui contiennent des fonctions de calcul et utilisent d'autres classes en interne.
— `Patient`, `Medecin`, `Medicament` et `LignePrescription` ont moins besoin de tests, elles stockent uniquement des données, seules les fonctions *getter* et *setter* sont éventuellement à tester pour augmenter la couverture de code.

Toutes les fonctions des classes dans le paquetage `service` sont à tester.

Certains des tests devront utiliser la librairie `Mockito`, voici un exemple de test unitaire qui l'utilise :

```
1  // RendezVousTest.java
2  // Les imports ont été supprimés.
3
4  @ExtendWith(MockitoExtension.class)
5  class RendezVousTest {
6
7      @Mock
8      private Medecin medecin;
9
10     @Mock
11     private Patient patient;
12
```

```
13      @Test
14      void testCalculPrixConsultationAvecMocks() {
15          // Configuration des mocks
16          // Tarif horaire de €60, donc €1 par minute
17          when(medecin.getTarifHoraire()).thenReturn(60.0);
18          // Taux de remboursement de 70% (reste 30% à charge)
19          when(patient.getTauxRemboursement()).thenReturn(0.70);
20
21          // Création du RDV (on utilise les mocks)
22          LocalDateTime dateRdv = LocalDateTime.of(2025, 6, 10, 14, 0);
23          RendezVous rdv = new RendezVous(patient, medecin, dateRdv, 30,
                "Consultation test");
24
25          // Calcul attendu : (60 / 60) * 30 * (1 - 0.70) = 1 * 30 * 0.3
                = 9.0
26          double prix = rdv.prixConsultation();
27
28          assertThat(prix).isCloseTo(9.0, within(0.01));
29      }
30  }
```

Un autre exemple de test unitaire qui utilise `Mockito` dans le fichier `GestionRendezVousTest.java`.

```
1   @ExtendWith(MockitoExtension.class)
2   class GestionRendezVousMockitoTest {
3
4       @Mock
5       private Medecin medecin;
6
7       @Mock
8       private Patient patient;
9
10      // GestionRendezVous est une classe de service
11      // C'est une classe concréte à tester
12      @InjectMocks
13      private GestionRendezVous gestionRendezVous;
14
15      @Test
16      void testPrendreRendezVousSucces() {
17          when(medecin.getNomComplet()).thenReturn("Jean Dupont");
18          LocalDateTime dateRdv = LocalDateTime.of(2025, 6, 10, 15, 0);
19
20          // On teste la création
21          RendezVous rdv = gestionRendezVous.prendreRendezVous(
22              patient, medecin, dateRdv, 20, "Suivi");
23
24          assertThat(rdv).isNotNull();
25          assertThat(rdv.getMedecin()).isEqualTo(medecin);
26          assertThat(rdv.getPatient()).isEqualTo(patient);
27          assertThat(gestionRendezVous.getNombreRendezVous()).isEqualTo
```

```
                (1);
28      }
29
30      @Test
31      void testPrendreRendezVousEchecSiIndisponible() {
32          when(medecin.getNomComplet()).thenReturn("Jean Dupont");
33
34          LocalDateTime dateRdv = LocalDateTime.of(2025, 6, 10, 16, 0);
35
36          // On ajoute un premier RDV
37          gestionRendezVous.prendreRendezVous(
38              patient, medecin, dateRdv, 30, "RDV 1");
39
40          // On essaie d'ajouter un second RDV au même moment pour le mê
                me médecin
41          RendezVous rdv2 = gestionRendezVous.prendreRendezVous(
42              patient, medecin, dateRdv, 30, "RDV 2");
43
44          assertThat(rdv2).isNull();
45          assertThat(gestionRendezVous.getNombreRendezVous()).isEqualTo
                (1);
46      }
47
48  }
```

Vous devez ajouter des méthodes de tests pour garantir une couverture de code par les tests supérieure à 80% pour les classes permettant la gestion des patients, des médecins, des rendez-vous, des médicaments et des ordonnances, ainsi que des fonctionnalités de recherche.

Il est rappelé que les tests doivent être écrits en respectant les bonnes pratiques de développement, c'est-à-dire :

- un test par méthode (éventuellement plusieurs tests pour une méthode si nécessaire ou utilisation des tests paramétrés pour parcourir l'ensemble des conditions) ;
- un test par exception ;
- chaque test doit être indépendant des autres tests (pas de dépendance entre les tests) et doit être autonome (utilisation des mocks quand c'est nécessaire).

**Exercice 6 :**

1. Écrivez les tests unitaires pour atteindre un minimum de 80% de couverture du code. Pour rappel, chaque collaborateur du groupe doit écrire au moins une classe de tests.

### Gestion de la documentation

Dans le projet, il est important de documenter le code source pour faciliter sa compréhension et sa maintenance. Il est recommandé d'utiliser des commentaires *Javadoc* pour documenter les classes, les méthodes et les attributs de votre code source.

**Exercice 7 :**

1. Ajoutez des commentaires Javadoc pour chaque classe, chaque méthode et chaque attribut de votre code source.
   — Les commentaires doivent être clairs, concis et pertinents.
   — Ils doivent expliquer le rôle de la classe, de la méthode ou de l'attribut, les paramètres d'entrée, les valeurs de retour, les exceptions levées, *etc.*
2. Testez que votre code source est correctement documenté en utilisant un outil de génération de documentation *Javadoc* (par exemple, en utilisant la tâche `javadoc` de Gradle).

### Réalisation de l'interface graphique (travail à réaliser l'après-midi)

Le but de cette partie est de développer une interface graphique en *JavaFX* pour votre application de **Système de gestion de dossiers médicaux**.

Pour que vos classes puissent fonctionner correctement avec *JavaFX*, vous allez devoir appliquer quelques modifications au code fourni pour vous assurer qu'il respecte les pré-requis et bonnes pratiques de *JavaFX*.

**Exercice 8 :**

1. Organisez les différentes classes de l'application pour qu'elles respectent le découpage *Modèle-Vue-Contrôleur*.
   Vous veillerez à placer les différents types dans les paquetages appropriés, à définir les interfaces nécessaires et à correctement identifier les différentes façades utilisées pour la manipulation du modèle.
   **Parmi les classes fournies, certaines peuvent déjà jouer le rôle de façade : vous devez indiquer dans les commentaires de ces classes qu'elles jouent ce rôle, et les completer si nécessaire.**
2. Dans certaines classes, des collections d'objets sont stockées sous la forme d'instances de `List`.
   Modifiez ces classes pour qu'elles utilisent à la place des `ObservableList`. Vous adapterez le code des méthodes existantes pour tenir compte de cette modification. Vous ajouterez également un *getter* pour ces `ObservableList`. Si un *getter* existe déjà pour cet attribut, vous devrez le modifier pour que son type de retour soit une `ObservableList`

### Affichage des différents éléments de l'application

Pour cette partie, vous allez devoir programmer un certain nombre de vues et de contrôleurs, afin de pouvoir afficher les différents éléments de votre application, notamment :

— la liste des `Medecin` de la clinique ;
— la liste des `Patient` de la clinique ;
— la liste des `Médicaments` donnés par les médecins aux patients ;
— la liste des `RendezVous` donnés par les médecins aux patients ;
— la liste des `Ordonnance` donnés par les médecins aux patients ;
— une vue principale permettant de naviguer entre les vues précédentes.

Pour chacun des éléments de cette liste, vous devez proposer une vue de type *Master-Detail*, avec la liste d'un côté et le détail de l'objet sélectionné de l'autre, en suivant les étapes décrites ci-après. Notez que les vues graphiques (et le code associé) à ces différents éléments sont totalement indépendantes, à part pour la dernière vue.
**De ce fait, vous pouvez et devez vous répartir le travail de sorte que chacun des membres du groupe réalise l'une de ces vues et le code associé. La vue principale doit obligatoire être réalisée, et chaque membre du groupe doit réaliser au moins une des autres vues (vous ne serez pas**

**pénalisés si toutes les vues ne sont pas réalisées tant que vous respectez ces consignes).**

**Exercice 9 :**

En suivant les consignes décrites dans les sections ci-dessous, vous devez :

1. Concevoir une vue `FXML` permettant d'afficher les éléments de l'application sous la forme d'une vue *Master-Detail*, et programmer le contrôleur associé à cette vue.
2. Définir des classes `Application` intermédiaires afin de pouvoir tester les différentes vues développées.
3. Concevoir une vue `FXML` permettant d'ajouter ou modifier un élément de la liste, et programmer le contrôleur associé à cette vue.
4. Permettre de naviguer entre la vue *Master-Detail* et la vue d'ajout/modification.

**Réalisation d'une vue *Master-Detail*** Pour réaliser une vue permettant de visualiser l'une des listes de l'application, vous devez :

— Créer un fichier FXML dédié à cette vue, pour lequel vous devez définir un `fx:controller`.
— Placer d'un côté de la vue un composant de type `ListView`, et de l'autre l'ensemble des composants nécessaires pour afficher les détails de l'élément actuellement sélectionné dans la liste.
— Affecter un `fx:id` à tous les composants qui en ont besoin.

**Attention :** Ne prenez pas trop de temps à concevoir cette vue. L'essentiel est qu'elle soit complète et fonctionnelle. Ce n'est pas grave si elle n'est pas « jolie ».

**Implémentation du contrôleur** Une fois votre vue réalisée, vous allez devoir implémenter le contrôleur associé. Pour cela, suivez les instructions suivantes :

— Générez le code du contrôleur à l'aide du *SceneBuilder*.
— Ajoutez à votre contrôleur une méthode permettant de stocker une instance de l'objet à afficher (par exemple, la liste des `Patient`) dans un attribut de votre contrôleur. Complétez cette méthode pour que la `ListView` de votre vue affiche les différents éléments (par exemple, les patients contenus dans la liste) et pour afficher les informations de l'élément sélectionné dans les composants dédiés.
— Par défaut, c'est la méthode `toString()` des éléments de la liste qui est utilisée pour l'afficher dans la `ListView`. Si cette méthode produit trop d'informations, ajoutez à la méthode

définie à la question précédente le code suivant, en l'adaptant selon vos besoin. Ici, l'information affichée pour un `Patient` dans la liste est uniquement son nom.

```
myPatientListView.setCellFactory(list -> {
  return new ListCell<>() {
    @Override
    public void updateItem(Patient patient, boolean empty) {
        super.updateItem(patient, empty);
        if (empty || (patient == null)) {
          setText(null);
        } else {
          setText(patient.getNom());
        }
      }
    };
});
```

Une fois le contrôleur implémenté, créez une classe principale *JavaFX* (par exemple en reprenant le code de `HelloApplication`). Cette classe principale doit charger la vue que vous avez créée, et associer au contrôleur les éléments qu'il doit afficher (par exemple, en lui donnant une liste de `Patient` que vous aurez remplie au préalable). Testez votre application graphique pour vous assurer de son bon fonctionnement à l'aide de la *Run configuration Gradle* que vous avez définie plus tôt.

**Ajout, édition et suppression d'un élément de la liste** Afin de pouvoir modifier le contenu de la liste, vous allez maintenant ajouter les composants et code nécessaires pour pouvoir ajouter, modifier ou supprimer l'un des éléments affichés dans la liste.

- Ajoutez à la vue *Master-Detail* des boutons permettant de réaliser les actions mentionnées.
- Associez une méthode au bouton de suppression pour que celle-ci récupère l'objet sélectionné dans liste puis le supprime (par exemple, en supprimant un `Patient` sélectionné de la liste).
- Associez aux deux autres boutons des méthodes qui devront charger une vue que nous définissons ci-après.

L'édition et l'ajout d'un nouvel élément dans la liste se feront à l'aide d'une vue dédiée. Suivez les instructions suivantes pour pouvoir la créer :

- Créer un fichier FXML dédié à cette vue, pour lequel vous devez définir un `fx:controller`.
- Ajoutez-y les composants permettant à l'utilisateur de saisir les informations relatives à l'élément à ajouter. Pour les champs demandant à l'utilisateur de choisir parmi un ensemble restreint de valeurs (par exemple, pour choisir un `Medecin`), vous utiliserez des `ComboBox` (vous pouvez en retrouver une description dans le cours et ci-dessous).
- Ajoutez également un bouton pour valider et un bouton pour annuler.
- Dans le contrôleur associé, ajoutez des méthodes permettant d'accéder à la collection des objets à modifier (par exemple, à une liste de `Patient`), pour que ces méthodes stockent la col-

lection dans un attribut.
- Dans la méthode associée au bouton pour valider, ajoutez le code nécessaire pour créer un nouvel objet et l'ajouter à la collection.
- Ajoutez une méthode permettant de stocker dans un attribut un (éventuel) objet préexistant dans votre contrôleur. Cette méthode préremplira les différents champs pour permettre d'éditer l'objet en question.
- Complétez la méthode de validation pour modifier cet objet s'il existe (c'est-à-dire, si l'attribut n'est pas `null`) au lieu d'en créer un nouveau.
- Dans le cas de l'édition, vous devrez forcer le rafraîchissement de la `ListView` : ajoutez une méthode permettant de stocker cette liste, et appelez sa méthode `refresh()` après avoir modifié l'objet sélectionné. Vous pouvez également utiliser la méthode `set(int index, T elt)` fournie par l'interface `List` pour écraser l'élément précédent et ainsi réaliser implicitement le rafraîchissement de la vue.

Afin de pouvoir gérer vos `ComboBox`, voici quelques informations utiles dont vous pourrez tenir compte dans votre code.

- Les `ComboBox` fonctionnnent globalement comme des `ListView`.
- Par défaut, le *SceneBuilder* vous génère des attributs de type `ComboBox<?>` : pour pouvoir les utiliser, vous devrez remplacer le `?` par le type des éléments affichés par la `ComboBox` (par exemple, `Medecin`, `String`, ...).
- Pour écouter une sélection d'élément dans la `ComboBox`, vous pouvez, comme pour les `ListView`, ajouter un écouteur sur la propriété `myComboBox.getSelectionModel().selectedItemProperty()` (par exemple pour activer ou désactiver des composants suivants la valeur saisie).
- Pour sélectionner programmatiquement un élément de la `ComboBox`, vous pouvez utiliser la méthode `myComboBox.getSelectionModel().select()`.

**Passage d'une vue à une autre** Pour l'instant, la vue affichant la liste et celle permettant de modifier l'un de ses éléments sont indépendantes. Il faut pouvoir passer de l'une à l'autre à l'aide des boutons et des méthodes associées. Suivez les étapes suivantes pour pouvoir le faire.

- Dans chacun de vos contrôleurs, définissez une méthode permettant de stocker dans des attributs une `Stage` et une `Scene`.
- Depuis la méthode `start()` de votre classe principale, appelez ces deux méthodes sur le contrôleur en lui fournissant la `Stage` et la `Scene` courante.
- Dans le contrôleur responsable de l'affichage de la liste, complétez les méthodes permettant d'ajouter ou de modifier un élément. Ces méthodes sont semblables à votre méthode `start()`, mais doivent charger la vue d'édition. Vous veillerez également à appeler les méthodes per-

mettant d'associer une `Stage` et une `Scene` à votre nouveau contrôleur (attention, la `Scene` à utiliser ici est celle définie en **attribut**, et non comme une variable **locale**). Vous associerez également à votre contrôleur la collection d'objets affichée (par exemple, la liste des `Patient`) et la `ListView` pour pouvoir la rafraîchir. Dans le cas d'une édition, vous associerez également votre nouveau contrôleur à l'élément actuellement sélectionné.

— Dans le contrôleur responsable de l'édition, ajoutez dans la méthode d'annulation le code permettant d'afficher dans la `Stage` définie en attribut la `Scene` également définie en attribut. Faites de même à la fin de la méthode de validation.

#### Création d'un « écran d'accueil »

Pour l'instant, votre application dispose de plusieurs vues permettant d'afficher la liste des patients, des médecins, *etc.* Cependant, ces vues sont pour le moment indépendantes : vous devez relancer votre application pour en changer. Pour éviter cela, nous allons définir une nouvelle vue, qui permettra d'accéder à toutes les autres.

**Exercice 10 :**

En suivant les consignes décrites ci-dessous, vous devez :

1. Concevoir une vue `FXML` permettant d'afficher l'écran d'accueil de l'application, et programmer le contrôleur associé à cette vue. Cet écran doit permettre de naviguer entre les différentes vues de l'application.
2. Définir une classe `Application` chargeant l'écran d'accueil et assurant la liaison entre les différentes vues et ce qu'elles doivent afficher.
3. Configurer le fichier `build.gradle` pour que ce soit cette classe qui soit considérée comme classe principale de votre application. **Assurez-vous que ce soit bien le cas dans la version finale de votre projet, pour que l'application qui sera lancée par votre évaluateur soit la bonne.**

Pour vous aider dans la réalisation de l'écran d'accueil, vous pouvez suivre les indications suivantes :

— Créez une nouvelle vue, comportant un bouton pour chacune des vues à afficher.
— Définissez un contrôleur pour cette vue, qui aura accès aux différents objets du modèle de votre application (par exemples, les `Patient`, les `Medecin`, *etc.*), au travers d'une façade que vous devrez identifier ou définir.
— Pour chacun des boutons de la vue, la méthode associée devra charger la vue correspondante, et associer à son contrôleur le(s) objet(s) dont il a besoin (par exemple, la liste des `Patient`

dans le cas de la vue qui affiche cette liste).
— Ajoutez un nouveau bouton dans chacune des vues existantes pour pouvoir revenir à l'écran d'accueil. Vous aurez pour cela besoin de stocker la `Scene` correspondant à cet écran d'accueil dans un nouvel attribut (vous ne pouvez pas réutiliser l'attribut de type `Scene` préexistant).

Une fois le contrôleur implanté, créez une nouvelle classe principale JavaFX (par exemple en reprenant le code de `HelloApplication`) qui se chargera d'afficher l'écran d'accueil défini ici. Cette classe principale doit charger la vue que vous avez créée, et associer au contrôleur tous les éléments du modèle dont il a besoin (par exemples, les `Patient`, les `Medecin`, *etc.*). Testez votre application graphique pour vous assurer de son bon fonctionnement.